# QZPaper Huggingface Daily

2026-03-03

1. **dLLM：简洁的扩散语言建模统一框架**
   dLLM 是首个统一扩散语言模型（DLM）开发全流程的开源框架，通过标准化训练、推理和评估三大核心组件，解决当前 DLM 领域的工程碎片化问题。框架还提供将预训练模型转换为 DLM 的端到端管线，支持小型 DLM 的可复现训练。
2. **From Scale to Speed：面向图像编辑的自适应测试时缩放**
   本文发现现有 Image-CoT 方法应用于图像编辑存在资源浪费、早期验证不可靠和结果冗余等问题。提出 ADE-CoT 框架，通过难度感知资源分配、编辑特定验证和深度优先停止三个策略，实现自适应测试时缩放，提升编辑效率。

# dLLM：简洁的扩散语言建模统一框架

Zhanhui Zhou*, Lingjie Chen*, Hanghang Tong, Dawn Song — UC Berkeley & UIUC, 2026

## 1. 动机与问题

扩散语言模型（DLMs）作为自回归语言建模的有前景替代方案，近年发展迅速，支持迭代精炼、灵活引导和高效解码。然而，当前 DLM 领域面临严重的**工程碎片化问题**：

- 多数开源 DLM（LLaDA、Dream 等）虽共享相似设计选择，但分散在各自的临时代码库中，缺乏透明实现，难以复现和对比；
- 不同模型暴露不一致的推理 API，推理算法难以跨模型复用；
- 评估流程因超参数敏感性差异极大——单一推理参数变化即可显著改变性能，而各论文使用不同评估工具和配置，统一评估困难；
- 缺乏将现有预训练模型（BERT、AR LM）转换为 DLM 的标准化流程。

dLLM 是首个**统一 DLM 开发全流程**的开源框架，标准化训练、推理和评估三大核心组件，同时保持足够灵活以支持新方法。框架还提供从零训练小型 DLM 的最小化可复现配方，包括将 BERT 编码器或自回归 LM 转换为 DLM 的端到端管线。

## 2. 数学基础

### 2.1 Masked Diffusion (MDLM)

给定离散 token 序列 $x = (x^1, \ldots, x^L) \in \mathcal{V}^L$，前向过程为吸收态掩码：

$$q(x_t^i | x_0^i) = (1-t)\mathbb{I}(x_t^i = x_0^i) + t\mathbb{I}(x_t^i = m)$$

其中 $m \notin \mathcal{V}$ 为掩码 token。模型 $p_\theta(x_0|x_t)$ 预测被掩码位置的原始 token，训练目标为带时间加权的负对数似然：

$$\mathcal{L}_{\text{MDLM}} = \mathbb{E}_{t\sim\mathcal{U}(0,1),x_0}\left[\frac{1}{t}\sum_{i\in\mathcal{M}_t} -\log p_\theta(x_0^i|x_t)\right]$$

权重 $1/t$ 平衡不同噪声水平的贡献。

### 2.2 Block Diffusion (BD3LM)

将序列分为 $K$ 个不重叠块 $B_1, \ldots, B_K$，块间自回归生成，块内扩散生成：$p_\theta(x) = \prod_{k=1}^K p_\theta(x^{B_k}|x^{<B_k})$。每个块的损失为：

$$\mathcal{L}_{\text{BD3LM}} = \sum_{k=1}^K \mathbb{E}_{t,x_0}\left[\frac{1}{t}\sum_{i\in\text{mask}(B_k,t)} -\log p_\theta(x_0^i|x_t^{B_k}, x^{<B_k})\right]$$

此分解允许复用前序块的 KV 缓存，兼得 AR 的缓存效率与扩散的并行解码优势。

## 3. 系统架构：三大核心组件

### 3.1 Trainer：统一训练接口

```
# MDLM PT -> SFT
trainer = MDLMTrainer(
  model=model,
  tokenizer=tokenizer,
  train_dataset=dataset["train"],
  eval_dataset=dataset["test"],
  args=training_args,
  data_collator=(
+   NoAttentionMaskWrapper(
      DataCollatorForSeq2Seq(
+       label_pad_token_id=eos_token_id,
      ),
+   )
  ),
)
trainer.train()
```

图 1：统一 *Trainer* 接口。*(a) MDLM* 预训练；*(b)* 一行切换为 *BD3LM*；*(c) MDLM SFT* 最小改动；*(d) AR-to-MDLM* 适配。

框架以 `MDLMTrainer` 和 `BD3LMTrainer` 为核心训练模块（`dllm/core/trainers`），支持预训练和微调。关键设计：

- **模块化**：切换 MDLM/BD3LM 预训练、MDLM SFT、AR-to-MDLM 适配仅需局部修改（换 Trainer、切换少量参数、包装 data collator），无需改变整体流程；
- **扩散逻辑与模型架构解耦**：新目标函数可最小化重构添加；框架还包含 EditFlow trainer 的参考实现（支持并行插入/替换/删除）；
- **基于 HuggingFace 生态**：使用 `accelerate`（支持 FSDP、DeepSpeed 分布式训练）、`peft`（参数高效微调），自定义 Trainer 为 `transformers.Trainer` 的轻量包装，简单易学且可扩展至大模型预训练。

### 3.2 Sampler：统一推理接口

不同 DLM 暴露不一致的推理 API。dLLM 引入轻量抽象 `Sampler(model).sample()`，将模型与推理算法解耦，支持即插即用的采样器替换。

**高效推理**。DLM 推理速度是实际瓶颈。框架集成 Fast-dLLM 加速 MDLM 解码：`MDLMFastdLLMSampler` 可作为标准 `MDLMSampler` 的直接替换。Fast-dLLM 包含两项关键优化：**CACHE**（块内近似 KV 缓存）和 **PARALLEL**（基于置信度的并行 token 更新），二者可组合使用。

**终端可视化器**。DLM 不同于 AR 模型的从左到右解码，token 可任意顺序生成。框架提供终端可视化器，展示 token 解码顺序和样本随解码步骤的演化过程，支持调试和可解释性分析。

### 3.3 Evaluation：统一评估流程

开源 DLM 依赖不同评估工具，且 DLM 对推理超参数**极度敏感**——单个参数变化（如是否抑制特定 token、CFG 值、并行度、温度）即可导致性能大幅下降。框架扩展 `lm-evaluation-harness`，精确匹配每个模型–任务对的预处理、解码设置和后处理配置，实现官方结果的忠实复现。

**关键发现**：DLM 评估对推理超参数高度敏感（如 max new tokens、temperature、CFG 等），这一事实在先前工作中很少被明确说明，但在复现实验中表现明显。

## 4. 开放配方与实验结果

### 4.1 微调大型开源 DLM 进行推理

使用 MDLM SFT + LoRA 在 s1K 数据集上微调 LLaDA 和 Dream 的 Base/Instruct 变体。训练配置：序列长度 4096，20 epochs，学习率 $10^{-5}$，全局 batch size 32，LoRA $r = 128$，$\alpha = 256$，权重衰减 0.1，余弦学习率 + 10% warmup，8×A100 + DeepSpeed ZeRO-2。

| 模型 | GSM8K | MATH | Count. | HEval |
|---|---|---|---|---|
| LLaDA-Inst | 79.9 | 34.8 | 19.9 | 35.4 |
| + SFT | **80.6** | **35.4** | **27.0** | **36.6** |
| Dream-Inst | 64.4 | 28.2 | 22.3 | 36.0 |
| + SFT | **70.4** | **32.8** | 20.3 | **37.2** |

Instruct 模型在数学、规划、代码基准上获得一致提升。Base 模型在域内数学任务（GSM8K、MATH500）上改进但在域外基准上退化。

### 4.2 BERT-Chat：将 BERT 转换为 DLM

以 ModernBERT（base/large）为骨干，仅通过 MDLM SFT（无需持续预训练或架构修改）在 Tulu 3 SFT + SmolTalk 混合数据上微调。训练配置：序列长度 1024，10 epochs，学习率 $10^{-4}$，全局 batch size 384，bf16，8×A100。

| 模型 | GSM8K | BBH | MATH | MMLU | Hella. |
|---|---|---|---|---|---|
| MBERT-base-chat | 3.6 | 21.1 | 3.1 | 26.2 | 34.5 |
| MBERT-large-chat | 9.3 | **25.6** | **3.6** | 29.6 | 40.9 |
| Qwen1.5-0.5B-Chat | 11.3 | 18.2 | 2.1 | 35.0 | 36.9 |
| GPT-2-medium | 2.1 | 17.8 | 1.4 | 22.9 | 39.4 |

ModernBERT-large-chat 在 BBH 和 MATH 上超越 Qwen1.5-0.5B-Chat，在多数基准上超越 GPT-2 变体——

尽管它是纯编码器模型且未做任何架构修改。这表明 BERT 风格骨干是 DLM 的可行起点。

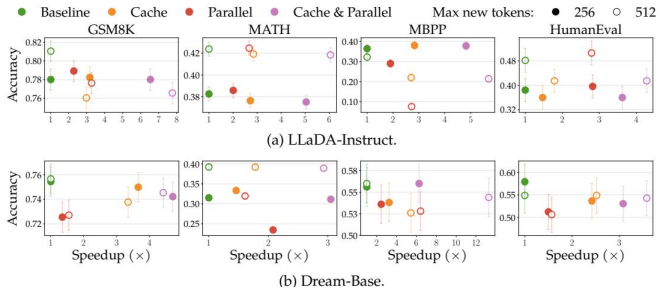
### 4.3 Tiny-A2D：将自回归 LM 转换为 DLM

以 Qwen3-0.6B 为骨干，分别用 MDLM 和 BD3LM 目标在指令跟随数据上 SFT（无持续预训练）。训练配置：MDLM 序列长度 1024、BD3LM 序列长度 512 + 块大小 32，10 epochs，学习率 $10^{-4}$，全局 batch size 2048，64×A100。实验中**未**使用 logits right shifting 技巧（与先前工作不同，因其导致性能退化）。

| 模型 | GSM8K | MATH | MMLU | HEval | MBPP |
|---|---|---|---|---|---|
| Qwen3-0.6B-mdlm | 29.3 | 8.7 | 40.0 | 30.5 | 29.2 |
| Qwen3-0.6B-bd3lm | 46.3 | 12.9 | 39.1 | **46.3** | **38.2** |
| Qwen3-0.6B-Base | 59.6 | 32.4 | 52.8 | 32.3 | 36.6 |

BD3LM 变体在代码生成（HumanEval 46.3、MBPP 38.2）上**超越**原始 Qwen3-0.6B-Base，尽管仅用 SFT 训练。两个 DLM 变体在知识和推理基准（MMLU、BBH）上仍落后 AR 对应版本，但验证了 AR-to-Diffusion 转换是构建 DLM 的实用且计算高效的路径。

### 4.4 Fast-dLLM 推理加速




(a) LLaDA-Instruct.

(b) Dream-Base.

图 2: *Fast-dLLM 在 max new tokens @ 256/512 下的评估结果。CACHE + PARALLEL 组合可达 5–11× 加速。*

在 LLaDA-Instruct 上，max new tokens @ 512 时：GSM8K 基线 81.1 acc / 6.7 tok/s，+Cache&Parallel 达 76.6 acc / 7.8× 加速；HumanEval 基线 48.2 acc，+Both 达 41.5 acc / 4.3× 加速。Dream-Base 上，MBPP @ 512 +Both 达 13.3× 加速且精度几乎不降（54.4 vs 56.0）。复现结果与官方 Fast-dLLM 实现一致。

### 4.5 评估复现验证

框架对 LLaDA 和 Dream 的官方结果进行了全面复现。LLaDA-Base 上：MMLU 65.9 vs 65.9（完全匹配），GSM8K 70.7 vs 70.3，MATH 32.4 vs 31.4。Dream-Base 上：MMLU 70.0 vs 69.5，BBH 63.7 vs 57.9（dLLM 更高），GSM8K 77.0 vs 77.2。整体复现结果与官方高度一致，少量差异来自必要的推理配置微调。

## 5. 相关工作

**离散扩散文本生成**。从 D3PM 的吸收态/均匀态扩散到连续时间扩展、基于 score 和 ratio 的目标，再到 MDLM 简化为独立 token 掩码。其他方向包括嵌入空间连续扩散、flow matching 与 edit-based 方法、Block Diffusion（AR 与扩散的插值）、混合 AR-扩散架构。

**开源大型 DLM**。从 Nie 等人首次将 MDLM 扩展到 1.1B，到 DiffuGPT/DiffuLLaMA（127M–7B AR 转 DLM）、RND1（30B）、LLaDA2.0（100B），再到 Dream（基于 Qwen-2.5 适配）和 LLaDA（8B MDLM 从零训练，性能接近 LLaMA3-8B）。商业系统如 Mercury 进一步验证 DLM 的生产可行性。

**DLM 工具缺口**。先前 DLM 缺乏统一开发流程；高效推理工具（Fast-dLLM）独立于训练和评估开发；评估流程因任务集和超参不同而不一致。dLLM 通过模块化 Trainer、即插即用 Sampler 和可复现评估流程填补了这一空白。

## 6. 讨论与局限

- **核心贡献**：首个统一 DLM 训练/推理/评估的开源框架；提供从 BERT 和 AR LM 转换为 DLM 的最小化配方；揭示 DLM 评估的超参数敏感性问题；
- **实用价值**：框架已支持 LLaDA、Dream 等主流模型的复现、微调和部署，所有模型检查点开源于 dllm-hub；
- **转换效率**：实验证明现有预训练模型（BERT 编码器和 AR LM）仅通过轻量 SFT、无需架构修改或持续预训练即可转换为功能性 DLM，大幅降低 DLM 开发门槛；
- **当前局限**：框架目前聚焦 MDLM 和 BD3LM 两大目标函数，尚未覆盖所有扩散目标；DLM 的 RL 算法尚不成熟，待广泛采用后集成；小模型配方在知识/推理基准上仍与 AR 模型存在差距；
- **未来方向**：随领域发展持续集成新方法（如 DLM 专用 RL 算法）；支持更多开源模型发布；进一步优化推理效率。

# From Scale to Speed：面向图像编辑的自适应测试时缩放

Qu*, Yuan*, Tang‡, Chen, Tang, Yu, Sun, Bai, Chu, Gou§, Xiong, Cai — 中科院信工所 / 阿里高德 / 昆士兰大学, 2025

## 1. 动机与问题定义

Image Chain-of-Thought (Image-CoT) 是一种测试时缩放范式，通过在推理时生成多个候选并选优来提升图像生成质量。现有 Image-CoT 方法主要面向**文本到图像 (T2I) 生成**——这是一个开放式任务，受益于大规模采样。然而**图像编辑是目标导向任务**：解空间受源图像与编辑指令双重约束，即使变换噪声或改写提示词，候选多样性仍然有限。

将 T2I 的 Image-CoT 直接迁移至编辑任务暴露三个核心问题：

- **资源分配低效**：现有方法对所有编辑使用固定采样预算（如 $N=32$）。实验显示简单编辑（初始分数高）几乎无提升，困难编辑获益显著，固定预算在简单场景浪费大量算力；
- **早期验证不可靠**：现有剪枝方法依赖通用 MLLM 分数评估中间去噪状态。编辑往往修改局部细微区域，早期阶段难以分辨，导致 **40%** 早期低分样本实际最终获得高分却被错误剪枝；
- **冗余编辑结果**：大规模采样常生成多个相同最高分的正确结果。例如最佳分数在 $[7,9)$ 范围的编辑案例中，超过 15 个候选共享相同最佳分数，但编辑任务仅需一个意图对齐的结果。

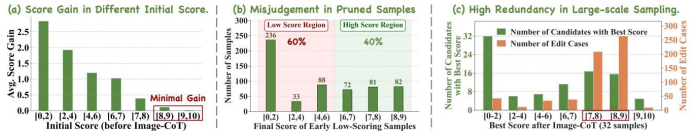



图 1：现有 *Image-CoT* 方法应用于编辑的三个问题。*(a)* 固定预算在简单编辑上浪费算力；*(b) 40%* 早期低分样本最终高分却被错误剪枝；*(c)* 大量冗余正确输出。

## 2. ADE-CoT 框架

为解决上述问题，本文提出 **ADaptive Edit-CoT (ADE-CoT)**，一个按需测试时缩放框架，核心思想是**从规模转向速度**（From Scale to Speed）。框架包含三个策略：难度感知资源分配、编辑特定验证的早期剪枝、深度优先机会性停止。

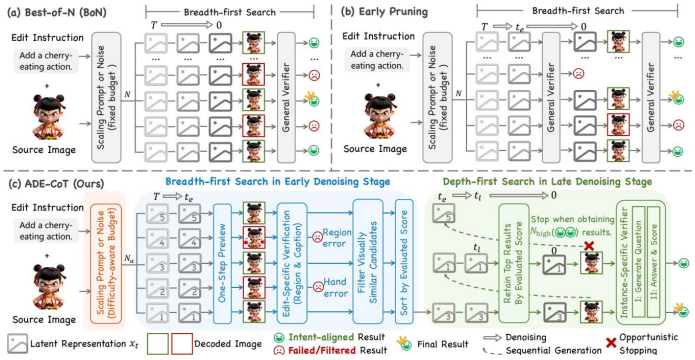



图 2：流程对比。*(a) Best-of-N* 宽度优先固定预算；*(b)* 已有剪枝用通用分数；*(c) ADE-CoT* 三策略协同：橙色 = 难度感知预算，蓝色 = 编辑特定验证，绿色 = 深度优先停止。

**预备知识**。给定源图 $I_{\text{src}}$ 和编辑指令 $c$，标准 Best-of-$N$ 方法生成 $N$ 个候选 $\{I_1,\ldots,I_N\}$（通过变换初始噪声），用验证器 $\text{Vrf}: I_{\text{src}} \times I \times c \to \mathbb{R}$ 打分，选最高分作为输出 $I^* = \arg\max_{I_i} \text{Vrf}(I_{\text{src}}, I_i, c)$。Early Pruning 在时间步 $t_e < T$ 生成中间预览，剪除低于阈值 $S_{\text{rj}}$ 的候选。

### 2.1 难度感知资源分配

ADE-CoT 首先生成单个候选并用验证器评分，获得初始分数 $S$ 作为编辑难度代理。自适应预算公式为：

$$N_a = N_{\min} + \lceil (N - N_{\min}) \times (1 - S/S_{\max})^{\gamma} \rceil$$

其中 $N_{\min}$ 为最小预算，$N$ 为原始预算，$S_{\max}$ 为最大可能分数，$\gamma$ 控制敏感度。当 $S \to S_{\max}$（简单编辑），$N_a \to N_{\min}$；当 $S \to 0$（困难编辑），$N_a \to N$。默认 $N_{\min}=1, \gamma=0.15, S_{\max}=10$。实验表明 $\gamma$ 从 0 增至 0.15 时 NFE 稳步下降而性能几乎不变。

### 2.2 编辑特定验证的早期剪枝

包含三个组件：单步预览机制、编辑特定验证器、视觉相似性过滤。

**单步预览机制**。由于近期编辑模型（Step1X-Edit、BAGEL、Kontext）主要基于 flow matching 训练，可从早期噪声潜变量 $x_{t_e}$ 单步估计干净潜变量：

$$x_{0|t_e} = x_{t_e} - \sigma_{t_e} \epsilon_\theta(x_{t_e}, T_{t_e})$$

解码后得到预览图 $I_{0|t_e}$，无需额外去噪步骤。相比 TTS-EF 需要额外去噪步或直接解码噪声潜变量，本方法在早期即可获得清晰预览且 NFE 最低。

**编辑区域正确性验证** $S_{\text{reg}}$。编辑失败的主要原因是编辑位置错误。先用 MLLM 识别待修改对象，再由 Grounded SAM2 生成二值掩码 $M \in \{0,1\}^{H\times W}$。计算逐像素变化图 $\Delta = \frac{1}{C}\sum_{c=1}^{C} |I^{(c)} - I_{\text{src}}^{(c)}|$，通过 softmax 归一化后聚合掩码内变化：

$$S_{\text{reg}} = \sum_{H,W} M \odot \text{softmax}_{H,W}(\Delta)$$

高 $S_{\text{reg}}$ 表明变化集中在预期区域。

**指令-描述一致性** $S_{\text{cap}}$。用 MLLM 根据 $I_{\text{src}}$ 和指令 $c$ 生成目标描述 $c_{\text{cap}}$，再用 CLIP 计算一致性：$S_{\text{cap}} = \text{CLIPScore}(I, c_{\text{cap}})$。

**统一评分与过滤**。三项分数加权融合：

$$S = S_{\text{gen}} + \lambda_{\text{reg}} S_{\text{reg}} + \lambda_{\text{cap}} S_{\text{cap}}$$

默认 $\lambda_{\text{reg}}=1, \lambda_{\text{cap}}=3$。低于阈值 $S_{\text{rj}}=5$ 的候选被剪枝。实验表明编辑特定分数将高分区域误判从 235 降至 86（降低 63%）。此外用 DINOv2 提取视觉嵌入，当两个预览相似度超过 $\tau_{\text{sim}}=0.9$ 时丢弃低分者，去除冗余候选。最终按 $S$ 降序排列，高早期分数的候选倾向获得更高最终分数。

### 2.3 深度优先机会性停止

剩余候选按早期分数**顺序处理**（深度优先），而非并行生成所有候选。包含两个组件：

**后期保留过滤**。在较晚时间步 $t_l$（$t_e < t_l < T$）再次生成预览并计算统一分数，采用自适应阈值（相对当前最高分）动态剪枝不太可能最优的样本。

**实例特定验证器**。通用分数 $S_{\text{gen}}$ 常对多个候选给出相同高分，无法区分细微错误。本文设计两阶段问询：(1) 根据 $I_{\text{src}}$ 和 $c$ 生成 5 个 yes/no 问题（覆盖指令遵循、美学等）；(2) MLLM 逐一回答，计 yes 数作为 $S_{\text{spec}}$。全部 yes 表示意图对齐。当累计 $N_{\text{high}}=4$ 个意图对齐候选后即终止搜索。如图 7 所示，通用分数对两个候选给出相似高分，但实例特定验证器通过“肩膀是否朝前？”等针对性问题正确识别了错误候选。

**混合搜索策略**。早期阶段采用宽度优先（BFS）保留高潜力候选，后期阶段采用深度优先（DFS）避免冗余计算。消融实验证实此混合策略在性能-效率权衡上最优。

## 3. 实验设置

**评测基准**：(1) **GEdit-Bench**：真实用户编辑，用 GPT-4.1 + VIE-Score 评估语义一致性 (G SC)、感知质量 (G PQ)、综合分 (G O)；(2) **AnyEdit-Test**：覆盖局部/全局/隐式编辑，报告 $\text{CLIP}_{\text{im}}$、$\text{CLIP}_{\text{out}}$、DINO 距离；(3) **Reason-Edit**：复杂推理场景，报告 PSNR、LPIPS、CLIP Score。

**效率指标**：NFE（总去噪步数）；推理效率 $\eta = \frac{1}{M}\sum_{i=1}^{M} \sigma_i \cdot \frac{S^{(i)}}{S_{\max}} \cdot \frac{NT}{\text{NFE}^{(i)}}$（$\sigma_i = 1$ 当性能不低于 BoN）；结果效率 $\xi = \frac{1}{M}\sum_{i=1}^{M} \sigma_i \frac{\text{NFE}_{\min}^{(i)}}{\text{NFE}^{(i)}}$。

**编辑模型**：Step1X-Edit ($T=28$)、FLUX.1 Kontext ($T=28$)、BAGEL ($T=50$)。MLLM 用 Qwen-VL-MAX，通用分数用 VIE-Score。所有结果为三次运行均值。

## 4. 核心实验结果

### 4.1 与 SOTA 方法对比（$N=32$）

| 方法 (Kontext) | G O↑ | $\eta$↑ | $\xi$↑ |
|---|---|---|---|
| BoN | 6.641 | 0.66 | 0.12 |
| PRM | 6.517 | 1.13 | 0.27 |
| PARM | 6.563 | 0.77 | 0.29 |
| TTS-EF | 6.376 | 0.98 | 0.57 |
| **ADE-CoT** | **6.695** | **1.47** | **0.66** |

三个关键发现：**(1)** ADE-CoT 在所有模型和基准上均达到与 BoN 相当或更优的性能，同时推理效率 $\eta$ 提升超过 2×。结果效率 $\xi$ 在 GEdit-Bench/AnyEdit/Reason-Edit 上分别提升 4.9 × /2.7 × /2.9×。**(2)** PRM 和 PARM 因通用分数在早期预览上误判严重，性能反而低于 BoN。**(3)** TTS-EF 效率高但性能差，因为仅保留单个最优早期样本在大规模采样时不可靠。

在 BAGEL 上，ADE-CoT 达到 G O = 6.972（BoN 6.908），$\eta = 1.27$, 加速 1.8×；在 Step1X-Edit 上，G O = 7.196（BoN 7.157），$\eta = 1.45$, 加速 2.0×。跨不同预算（$N = 2, 4, 8, 16, 32$）的缩放曲线持续显示 ADE-CoT 以更低计算成本达到更高性能。

### 4.2 性能与效率分析

**难度感知分配有效减少简单编辑开销。**$\gamma$ 从 0 增至 0.15 时 NFE 稳步下降而 G O 几乎不变。超过 0.15 后性能开始下降。消融表（Tab.2 行 a）确认此策略在三个模型上分别将 NFE 从 896/1600/896 降至 797/1391/778。

**编辑特定验证器显著减少误判。**使用统一分数 $S$ 后，高分区域误判从 235 降至 86（63% 降低），同时低分剪枝数几乎不变（357 → 329）。当 $N > 8$ 时，仅用通用分数的累积误判导致性能低于 BoN，而 $S$ 始终保持优势。

**机会性停止有效减少冗余。**深度优先停止策略与 BoN/PRM/PARM 结合后均稳定减少 NFE 至少 10%，性能几乎无损。$N_{\text{high}} = 4$ 时性能饱和而 NFE 线性增长，故取 4 为默认值。

### 4.3 消融实验

**逐步消融（Kontext/GEdit-Bench）：**基线 BoN G O = 6.641, NFE = 896 → + 难度感知 6.641/797 → + 早期剪枝 ($S_{\text{gen}}$) 6.642/719 → + 编辑特定分数 ($S$) 6.647/673 → + 相似性过滤 6.651/508（NFE 降 24%，G O 不变）→ + 后期保留 6.652/464 → + 实例特定验证器 6.702/464（性能提升显著）→ + 机会性停止 6.695/418。完整 ADE-CoT 将 NFE 从 896 降至 418（53% 减少），同时 G O 从 6.641 提升至 6.695。

**预览方式消融：**额外去噪步 G O = 6.678/NFE = 523；直接解码噪声潜变量 6.648/790；单步预览（ADE-CoT）6.695/418。单步预览兼得清晰度与效率。

**搜索策略消融：**纯 BFS G O 最高但 NFE 最高；纯 DFS NFE 最低但 G O 偏低；混合策略（早期 BFS + 后期 DFS）$\eta$ 最高，实现最优性能-效率权衡。

**时间步选择：**Kontext 上 $t_e = 8$ 时预览清晰度足以准确剪枝，更大值仅增加成本；$t_l = 16$ 时预览已高保真，性能饱和。

**MLLM 验证器：**测试 Qwen2.5-VL-72B、Qwen-VL-MAX、Qwen3-VL-32B 三种 MLLM，ADE-CoT 在所有验证器上均实现 > 2× 加速。更强 MLLM（Qwen3-VL）带来更高 G O 和更大效率增益（Kontext 上 G O 6.719/NFE 403 vs BoN 6.691/896）。

## 5. 讨论与局限

ADE-CoT 的核心贡献在于识别 Image-CoT 应用于编辑的三个根本问题，并提出针对性解决方案。三个策略形成完整流水线：难度感知分配在宏观层面调节资源，编辑特定验证在早期阶段精准筛选，深度优先停止在后期阶段消除冗余。该框架作为即插即用模块，成功应用于三个不同架构的 SOTA 编辑模型，展现了良好的通用性。

**局限性：**(1) 编辑特定验证器依赖 Grounded SAM2 和 MLLM，对极端案例（如全局风格迁移无明确编辑区域）可能不适用；(2) 实例特定验证器的 yes/no 问题质量依赖 MLLM 能力，弱模型可能生成无关问题；(3) 当前框架聚焦单轮编辑效率，多轮编辑的级联优化尚待探索；(4) 难度估计基于单次采样，可能对边界案例不够鲁棒。未来方向包括将 ADE-CoT 扩展至视频编辑、3D 编辑等更广泛的目标导向生成任务，以及探索强化学习驱动的自适应停止策略。